



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

Facultad de Ciencias

Sistemas Operativos

Mecanismos de Entrada/Salida,
UART y Arquitectura de Controladores

Práctica 9

Presenta:

Rojo Peña Manuel Ianluck

Profesor:

Javier León Cotonieto

Ayudante laboratorio:

Adrián Martínez Manzo

Grupo: 7004, 2026-2

Fecha de entrega:

11 de mayo, 2026

Introducción

En esta práctica estudiaremos los tres mecanismos fundamentales mediante los cuales un sistema operativo puede coordinar la transferencia de datos entre el procesador y un periférico de **Entrada/Salida**: *Polling*, *Interrupciones* y *Acceso Directo a Memoria* (DMA). Lejos de ser una discusión puramente teórica, el objetivo central es desarrollar la capacidad de seleccionar el mecanismo correcto para un escenario concreto, justificando esa decisión con argumentos formales extraídos de la teoría de *Sistemas Operativos*.

Estructuramos la práctica en cuatro módulos progresivos:

- En el **Módulo 1** partimos de la intuición para formular ideas sobre las ventajas y desventajas de cada mecanismo, antes de haber consultado alguna fuente formal.
- En el **Módulo 2** confrontamos estas suposiciones con la bibliografía académica, corrigiendo errores y añadiendo los matices que la teoría formal introduce.
- En el **Módulo 3** nos enfocamos en el protocolo de hardware UART, que es el canal de comunicación serial utilizado en la implementación, describiendo su funcionamiento asíncrono y representando el proceso de recepción de un byte mediante un diagrama de flujo detallado.
- Finalmente, el **Módulo 4** integra todo lo anterior en un caso de estudio real, a partir de un correo lleno de ambigüedades y ruido de oficina, extraemos los requerimientos técnicos concretos, seleccionamos y justificamos el mecanismo de **E/S** óptimo, e implementamos la solución en C sobre la arquitectura Raspberry Pi Pico con el entorno de simulación Wokwi.

Módulos de Implementación y Análisis Arquitectónico

Módulo 1: Hipótesis sobre Mecanismos de Transferencia

Polling (Sondeo)

La ventaja intuitiva de este mecanismo es que el procesador siempre está pendiente de recibir una respuesta, por lo que en cuanto el dispositivo esté listo, la reacción es prácticamente inmediata, como bajar a preguntar al conserje y encontrarlo justo en ese momento. La desventaja, sin embargo, es evidente, dedicar al procesador a revisar constantemente un registro es un desgaste enorme de un recurso valioso. Por esa simple tarea de esperar, el CPU deja de poder hacer cualquier otra cosa.

Interrupciones (*Interrupts*)

El mecanismo podría salir mal cuando el procesador se encuentra en medio de una tarea que requiere ejecutarse de forma continua y sin pausa. Si llega una interrupción en ese momento, la tarea queda suspendida a la mitad, lo que podría dejarla en un estado inconsistente o causar errores por no haber terminado correctamente el trabajo antes de atender al timbre.

DMA (Acceso Directo a Memoria)

Aunque delegar toda la mudanza al asistente es muy eficiente, hay situaciones en las que no conviene esperar a que el camión esté completamente vacío para enterarse de lo que llegó. Algunos procesos necesitan saber cuándo se ha descargado cada elemento, ya sea porque son datos especiales que requieren verificación inmediata, o porque podría haber “cajas sospechosas” entre los datos recibidos, es decir, errores o datos corruptos que sería peligroso dejar pasar sin revisión hasta el final de la transferencia.

Módulo 2: Investigación y Corrección de Hipótesis

A continuación se presentan los tres mecanismos tal como los describe la referencia tomada de *Operating System Concepts*, décima edición, de Abraham Silberschatz, Peter Baer Galvin y Greg Gagne:

Polling

El protocolo básico de sondeo usa dos bits de estado: el *busy bit* (indica si el controlador está ocupado o libre) y el *command-ready bit* (indica si hay un comando disponible para procesar). El host coordina con el controlador mediante un protocolo de *handshaking*: espera a que el *busy bit* se limpie, escribe el comando y el byte de datos, activa el *command-ready bit*, y el controlador lo detecta, procesa el byte y limpia todos los bits al terminar.

En el paso de espera, el host realiza *busy-waiting*: lee el registro de estado en un bucle continuo hasta que *busy* se limpie. Si el controlador responde rápido, el

método es razonable en términos de ciclos gastados. En muchas arquitecturas, sondear el estado de un dispositivo requiere únicamente tres instrucciones de CPU, lo que lo hace eficiente en principio. Sin embargo, el sondeo se vuelve ineficiente cuando el dispositivo raramente está listo y existe procesamiento útil pendiente. En ese escenario, es preferible que el controlador notifique al CPU cuando esté disponible, mediante el mecanismo de interrupciones.

Contraste con la respuesta en el Módulo 1. En la respuesta propuesta en el módulo se acertó en identificar la disponibilidad inmediata como la ventaja central y el desgaste del procesador como la desventaja. Lo que no se anticipó es que ese desgaste no es absoluto, sino condicional, la referencia señala que sondear un dispositivo puede requerir tan solo tres instrucciones de CPU, y que si el controlador responde rápido, el costo es razonable. El error en la hipótesis fue asumir que el desperdicio siempre ocurre, cuando en realidad solo se vuelve problemático cuando el dispositivo raramente está listo y hay procesamiento útil alternativo que el CPU podría estar realizando.

Interrupciones (*Interrupt-driven I/O*)

El CPU dispone de una línea llamada *interrupt-request line* que revisa después de ejecutar cada instrucción. Al detectar una señal en esa línea, guarda su estado de ejecución actual y salta al manejador de interrupciones cuya dirección está almacenada en una tabla fija. El manejador determina la causa de la interrupción, la procesa y restaura el estado previo del procesador antes de retomar el flujo original.

Los sistemas modernos gestionan desde cientos hasta cientos de miles de interrupciones por segundo. Para hacerlo de forma controlada, se requieren capacidades avanzadas como las siguientes.

- **Enmascaramiento de interrupciones:** la capacidad de diferir o bloquear interrupciones mientras se ejecutan secciones críticas del kernel.
- **Vectorización eficiente:** un vector de interrupciones con direcciones de manejadores especializados, de modo que el sistema no tenga que sondear cada dispositivo para identificar la fuente. Cuando hay más dispositivos que entradas en el vector, se emplea *interrupt chaining*, donde cada entrada apunta a una lista de manejadores que se recorren hasta encontrar el adecuado.
- **Niveles de prioridad:** la mayoría de las CPU tienen al menos dos líneas de interrupción: una no enmascarable (reservada para errores irrecuperables) y una enmascarable (usada por controladores de dispositivos, que puede desactivarse durante secuencias críticas).
- **Soporte para traps de software:** un mecanismo que permite a las instrucciones solicitar la atención del sistema operativo directamente, utilizado en llamadas al sistema.

Para manejar la complejidad de los manejadores, muchos sistemas los dividen en dos fases: un manejador de primera instancia (FLIH, *First-Level Interrupt Handler*) que guarda el contexto y encola la operación, y un manejador de segunda instancia (SLIH, *Second-Level Interrupt Handler*) que ejecuta la operación de forma planificada. En sistemas como Solaris, estos manejadores se implementan como hilos del kernel con alta prioridad de planificación, permitiendo ejecución concurrente en arquitecturas multiprocesador.

Contraste con la respuesta en el Módulo 1. La idea identificó correctamente el caso donde el problema aparece, una tarea que no puede interrumpirse a la mitad, la referencia formaliza esa intuición con el concepto de sección crítica y añade la solución que el sistema operativo tiene para ese problema, que es el enmascaramiento de interrupciones. Lo que no contemplamos en la idea fue que los sistemas modernos ya resuelven de forma estructurada el riesgo de interrumpir trabajo en curso, mediante niveles de prioridad, vectorización y la separación en manejadores de primera y segunda instancia (FLIH y SLIH). La intuición fue correcta en señalar el problema, pero subestimamos la sofisticación del ecosistema que existe precisamente para mitigarlo.

Acceso Directo a Memoria (DMA)

Transferir datos byte a byte a través del CPU (modelo conocido como *Programmed I/O*) resulta ineficiente para dispositivos de alto volumen como discos. El controlador DMA descarga ese trabajo del procesador.

Para iniciar una transferencia, el host escribe un bloque de comandos DMA en memoria que contiene tres elementos: un puntero al origen de los datos, un puntero al destino, y la cantidad de bytes a transferir. El método *scatter-gather* permite especificar múltiples transferencias no contiguas dentro de un solo comando. El CPU escribe la dirección de ese bloque al controlador DMA y queda libre para continuar con otras tareas.

El DMA opera el bus de memoria directamente, sin intervención del CPU. El protocolo de coordinación entre el controlador DMA y el del dispositivo usa dos señales: *DMA-request* (dato disponible para transferir) y *DMA-acknowledge* (confirmación de que la transferencia ocurrió). Al finalizar la transferencia completa, el DMA interrumpe al CPU para notificarlo.

Durante la operación, el CPU pierde acceso momentáneo a la memoria principal en cada ciclo que el DMA necesita el bus (*cycle stealing*), aunque conserva acceso a su caché interna. Este costo generalmente se justifica por la mejora en el rendimiento global del sistema.

Un inconveniente adicional tiene que ver con la protección de memoria. Dado que el destino ideal de una transferencia DMA es el espacio del kernel (el espacio de usuario podría modificarse durante la transferencia), a menudo se requiere una

segunda copia de datos del espacio del kernel al espacio del usuario, lo que resulta en un *doble buffering* ineficiente. Por esa razón, los sistemas operativos modernos tienden a emplear *memory-mapping* para transferir datos de E/S directamente entre el dispositivo y el espacio de usuario. Algunas arquitecturas van más lejos e implementan DVMA (*Direct Virtual Memory Access*), que puede transferir entre dos dispositivos mapeados en memoria sin intervención del CPU ni uso de memoria principal.

Contraste con la respuesta en el Módulo 1. Nuestra idea apuntó a algo válido, hay casos en los que no conviene esperar al final de una transferencia para inspeccionar los datos, porque podrían llegar errores o información que requiere atención inmediata. Sin embargo, la referencia muestra que esa no es la razón principal por la que DMA no se usa en todas las transferencias, las razones formales tienen más que ver con el costo de configuración del canal DMA (que supera el beneficio en transferencias pequeñas), con la contención del bus de memoria durante el *cycle stealing*, y con la complejidad que introduce el doble buffering cuando el destino debe ser el espacio del usuario. Con nuestra respuesta intuimos un problema real de validación de datos, pero la teoría formal lo ubica en un segundo plano frente a los costos de arquitectura que sí son los trade-offs determinantes.

Módulo 3: Protocolo Serial UART y Diagrama de Flujo

¿Qué es UART?

UART (*Universal Asynchronous Receiver-Transmitter*) es un protocolo de comunicación serial asíncrono ampliamente utilizado en sistemas embebidos para intercambiar datos entre dos dispositivos a través de una línea de transmisión. La palabra clave en su nombre es *asíncrono*, a diferencia de los protocolos síncronos como SPI o I²C, UART no utiliza una señal de reloj compartida entre el transmisor y el receptor, en su lugar, ambos dispositivos se ponen de acuerdo de antemano en una velocidad de transmisión común, y cada uno usa su propio reloj interno para sincronizarse durante la recepción de cada byte.

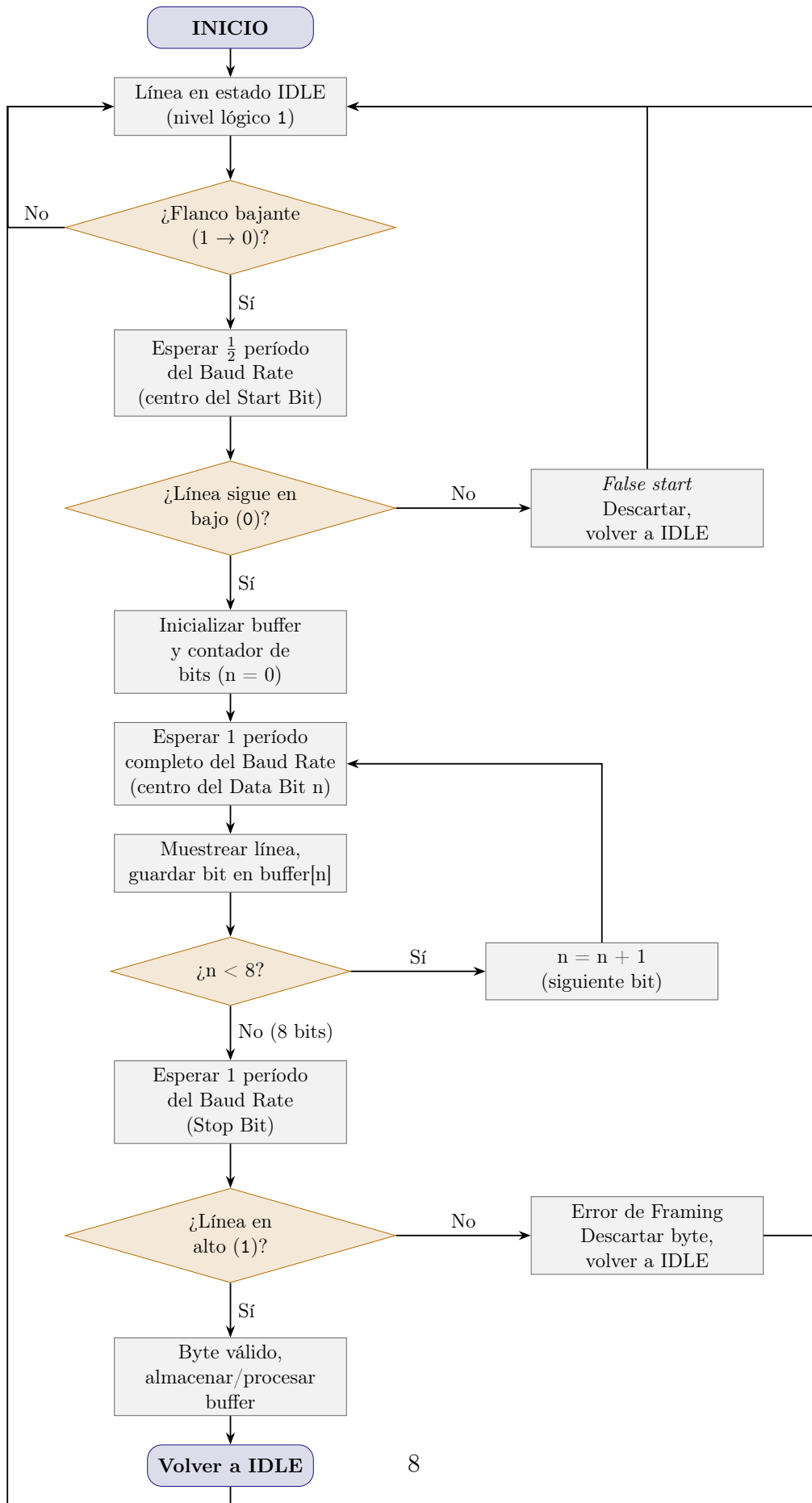
Conceptos clave del protocolo

- **Baud Rate.** El *baud rate* es la velocidad de transmisión, expresada en bits por segundo (bps), determina cuánto tiempo dura cada bit en la línea, por ejemplo, con una velocidad de 9600 baudios, cada bit ocupa aproximadamente 104 microsegundos; con 115200 baudios (la velocidad más común en sistemas modernos), cada bit dura aproximadamente 8.68 microsegundos. Que ambos dispositivos usen exactamente el mismo *baud rate* es esencial, una diferencia pequeña puede acumular error suficiente como para corromper los datos al final de un byte.
- **Start Bit.** La línea UART se mantiene en estado alto lógico (1) cuando está en reposo, condición llamada *idle*, cuando el transmisor quiere enviar un byte, lo primero que hace es bajar la línea a estado bajo lógico (0) durante exactamente un período de *baud rate*. Esa transición de 1 a 0 es el *start bit*, y su función es alertar al receptor de que está por llegar un byte, el receptor usa esta transición como punto de sincronización para comenzar a muestrear los bits subsiguientes.
- **Data Bits.** Después del *start bit* vienen los bits de datos, que normalmente son 8, aunque el protocolo admite configuraciones de 5 a 9 bits, se transmiten en orden del bit menos significativo al más significativo (LSB primero), cada bit dura exactamente un período de *baud rate*, y el receptor los muestrea en el centro de ese período para maximizar el margen frente a ruido o ligeras diferencias de reloj.
- **Stop Bit.** Al finalizar los bits de datos, el transmisor regresa la línea al estado alto (1) durante uno o dos períodos de *baud rate*, ese es el *stop bit*, y tiene dos propósitos: confirmar al receptor que el byte llegó completo (si la línea no está en alto en este momento, se produce un error de *framing*), y garantizar un período mínimo de reposo antes del siguiente byte.
- **Sin bit de paridad (en la configuración más común).** Muchas implementaciones operan en modo 8N1, 8 bits de datos, sin paridad y 1 *stop bit*, es la configuración por

defecto en la mayoría de las herramientas y la que usa la plantilla de código de esta práctica.

Diagrama de flujo, Recepción de un byte UART

En el siguiente diagrama describimos paso a paso la lógica que debe seguir el hardware o software para recibir correctamente un solo byte a través de una línea UART, con los tiempos relativos al *baud rate*.



Módulo 4: Caso de Estudio

Análisis del correo, frases clave y su traducción técnica

1. *“deleguemos todo el movimiento de arreglos masivos de datos directamente hacia la memoria sin que el CPU intervenga”*

Traducción técnica: Esta frase describe el patrón de uso de DMA, pensado para transferencias de gran volumen donde el CPU no debe participar byte a byte. Sin embargo, el gerente nos aclara explícitamente que esto aplica al sistema final, no al prototipo de hoy. Por lo que fue considerada ruido técnico para el escenario inmediato, aunque útil para entender la arquitectura final.

2. *“reciba una clave de validación de exactamente 16 bytes por el puerto serial”*

Traducción técnica: La transferencia es pequeña, acotada y completamente conocida en tamaño, inicializar un canal DMA para mover 16 bytes introduce una sobrecarga de configuración totalmente desproporcionada, se favorecen mecanismos directos y simples como Polling.

3. *“solución más directa, básica y rudimentaria que puedan teclear en 45 minutos”*

Traducción técnica: Se prioriza la baja complejidad de implementación y el tiempo corto de desarrollo, lo que descarta cualquier mecanismo que requiera configurar subsistemas adicionales de hardware, como canales DMA o vectores de interrupción.

4. *“máquina host muy vieja y errática [...] a veces manda un par de caracteres, se pasma dos o tres segundos, y luego manda el resto”*

Traducción técnica: La entrada es lenta, irregular e impredecible en su temporización, lo que significa que el receptor necesita un mecanismo que pueda esperar indefinidamente entre bytes sin perder sincronía, lo cual es precisamente la fortaleza del Polling en modo bloqueante.

5. *“es un crimen tener al procesador congelado y bloqueado esperando [...] pero en este milisegundo exacto del encendido el chip no tiene absolutamente ninguna otra tarea asignada”*

Traducción técnica: El gerente reconoce el costo teórico del *busy waiting*, pero lo anula con el contexto, el CPU no tiene ninguna otra tarea que realizar durante el arranque. El costo de ocupar ciclos de CPU en Polling es cero desde la perspectiva del sistema, porque no hay trabajo alternativo que se esté perdiendo.

6. “*tienen que bloquear a nivel físico cualquier evento, alarma o señal asíncrona externa*”

Traducción técnica: Se deben deshabilitar las interrupciones a nivel del NVIC del procesador durante toda la fase de arranque, esta instrucción descarta explícita y completamente el uso de *Interrupt-driven I/O*.

7. “*el flujo de lectura debe ser lineal y blindado contra avisos externos*”

Traducción técnica: Se requiere ejecución secuencial, determinista y sin desvíos de flujo provocados por señales externas, es la descripción exacta de lo que ofrece Polling, un bucle de espera activa sin cambios de contexto, sin vectores de interrupción, sin handlers.

El único mecanismo coherente con la totalidad de los requerimientos anteriores es **Polling**, para defender esta decisión es necesario que transformemos las preocupaciones del gerente en conceptos de SO's.

El escenario que describe el correo corresponde a una fase de arranque seguro donde el sistema se encuentra en un estado privilegiado mínimo, no hay sistema operativo corriendo, no hay procesos planificados, no hay hilos del kernel activos, el único comportamiento esperado del procesador durante estos instantes es esperar y capturar exactamente 16 bytes.

En primer lugar, el correo prohíbe de forma explícita cualquier señal asíncrona externa, lo que equivale a deshabilitar el enmascaramiento de interrupciones en el NVIC, con las interrupciones bloqueadas, el mecanismo de *Interrupt-driven I/O* queda físicamente imposibilitado, el hardware simplemente no puede notificar al CPU de la llegada de datos. Polling, al ser un bucle de lectura activa sobre los registros del periférico, no depende de ninguna notificación externa y funciona correctamente con interrupciones deshabilitadas.

En segundo lugar, el principal argumento en contra del Polling es su costo en ciclos de CPU desperdiciados (*busy waiting*), no obstante, ese costo solo existe cuando hay otras tareas que podrían estar utilizando esos ciclos. Durante la fase de arranque descrita en el correo, no existe ninguna otra tarea, el procesador no tiene nada más que hacer, por lo que el *busy waiting* no desperdicia ningún recurso que pudiera aprovecharse de otra forma, el trade-off desaparece.

En tercer lugar, la transferencia de 16 bytes es un volumen de datos trivial, inicializar un canal DMA requiere configurar registros de dirección origen, dirección destino, contador de bytes y señales de control, todo ello más costoso en tiempo de desarrollo y en ciclos de configuración que simplemente leer 16 bytes en un bucle, el gerente lo expresa claramente al pedir que no se pierda tiempo inicializando subsistemas complejos.

Las interrupciones requieren que el NVIC esté habilitado y que exista un vector de interrupciones configurado con la dirección del manejador correspondiente, el auditor de seguridad nos indica que durante el arranque se deben bloquear a nivel físico todas las señales asíncronas externas, lo que en términos de arquitectura ARM equivale a ejecutar las instrucciones CPSID i o deshabilitar la IRQ correspondiente en el NVIC. Con esa restricción activa, el controlador

UART no puede generar interrupciones al CPU, por lo que el mecanismo queda inoperable para este escenario.

DMA por otro lado está pensado para mover bloques grandes de datos sin que el CPU intervenga byte a byte, lo que tiene sentido cuando esos bloques son de cientos o miles de bytes y el CPU tiene otras tareas más urgentes que atender durante la transferencia. En este caso, la transferencia es de 16 bytes, el CPU no tiene otras tareas y el objetivo es una implementación básica en 45 minutos, el costo de configurar un canal DMA es mayor que el problema que resolvería, el gerente lo descarta explícitamente al pedir que se ignore la sugerencia del arquitecto de software (que describía DMA) para esta etapa de arranque.

Mecanismo	Viable	Razón principal
Polling	Sí	Determinista, simple, funciona sin interrupciones
Interrupciones	No	Requieren IRQ habilitada; el auditor las prohíbe
DMA	No	Sobrecomplejo para 16 bytes; gerente lo descarta

Implementación Práctica en Wokwi

Una vez que el análisis del correo dejó claro que Polling es el único mecanismo coherente con todos los requerimientos del escenario, se procedió a completar la plantilla de código provista en el Anexo del manual. A continuación presentamos el archivo `main.c` completo con cada bloque TODO resuelto y comentado para dejar explícita la razón arquitectónica de cada decisión.

Sobre los TODO 1 y TODO 2.

La plantilla reservaba estos espacios para incluir librerías adicionales de hardware (como `hardware/irq.h` para Interrupciones o `hardware/dma.h` para DMA) y para definir una Rutina de Servicio de Interrupción (ISR), dado que la arquitectura elegida es Polling, ninguno de estos subsistemas se inicializa, y ambos bloques se dejan vacíos de forma intencional.

Sobre el TODO 3: inicialización del UART.

Se inicializa el periférico con `uart_init`, que configura los registros internos del controlador UART0 del RP2040 con el *baud rate* de 115200, luego se asignan los pines físicos 0 y 1 a su función de hardware UART mediante `gpio_set_function`, mapeando el pin 0 como TX y el pin 1 como RX, esta es la configuración mínima e indispensable para que el periférico pueda comunicarse con la máquina host.

Sobre el TODO 4: configuración del mecanismo de Polling.

Este bloque traduce directamente las dos restricciones técnicas más importantes del caso de estudio, la llamada a `irq_set_enabled(UART0_IRQ, false)` deshabilita en el NVIC (el controlador de interrupciones del núcleo ARM Cortex-M0+) la IRQ específica del periférico UART0, cumpliendo la instrucción del auditor de bloquear cualquier señal asíncrona a nivel físico. La llamada a `uart_set_fifo_enabled(UART_ID, false)` desactiva el buffer FIFO de hardware del UART, de modo que cada byte quede disponible para lectura de forma inmediata en el registro de datos, sin depender de un umbral de llenado que podría introducir

comportamiento no determinista en la lectura.

Sobre el TODO 5: bucle principal de recepción.

La lógica de recepción consiste en dos bucles anidados, el bucle exterior controla cuántos bytes se han capturado y no termina hasta tener exactamente `KEY_LENGTH` (16 bytes), el bucle interior es el núcleo del Polling: `uart_is_readable()` consulta el bit de estado del registro del periférico UART para saber si hay un byte disponible; mientras no lo haya, se ejecuta `tight_loop_contents()`, que en la arquitectura ARM se traduce en una instrucción NOP que mantiene al CPU ocupado sin efectos secundarios. Cuando el byte está disponible, `uart_getc()` lo lee del registro y lo almacena en `challenge_key`, el implementado eco mediante `printf` permite verificar en la terminal del simulador que cada carácter fue recibido correctamente:

```
1 #include <stdio.h>
2 #include "pico/stdlib.h"
3 #include "hardware/uart.h"
4
5 /* =====
6  TODO 1:
7  No incluimos bibliotecas adicionales dado que Polling no requiere
8  ni hardware/irq.h (Interrupciones) ni hardware/dma.h (DMA)
9  Ambos mecanismos los descartamos por el analisis del caso de estudio
10 ===== */
11
12 //Definicion de parametros del hardware
13 #define UART_ID uart0
14 #define BAUD_RATE 115200
15 #define UART_TX_PIN 0
16 #define UART_RX_PIN 1
17 //Requerimiento del caso de estudio: Exactamente 16 bytes
18 #define KEY_LENGTH 16
19 //Buffer global para guardar la clave
20 char challenge_key[KEY_LENGTH + 1];
21 //Variable de control (Util si tu implementacion es asincrona)
22 volatile bool key_received = false ;
23
24 /* =====
25  TODO 2:
26  No se define ninguna ISR pues con Polling y las interrupciones
27  del UART deshabilitadas, el hardware nunca generara una senial
28  de interrupcion, por lo que un handler aqui nunca seria invocado
29  ===== */
30
31
32 void
33 secure_boot_init()
34 {
35     //Inicializa la salida estandar (para ver los printf en consola)
36     stdio_init_all();
37 }
```

```

38  /* =====
39  TODO 3:
40  Inicializacion Basica del UART
41  Configuramos el periferico con el ID y el Baud Rate definidos
42  para despues mapear los pines fisicos a su funcion de hardware UART
43  =====
44  */
45  //Arrancamos el controlador UART0 del RP2040 con 115200 bps
46  uart_init(UART_ID, BAUD_RATE);
47
48  //Asignamos el pin 0 (TX) y el pin 1 (RX) a la funcion UART
49  gpio_set_function(UART_TX_PIN, GPIO_FUNC_UART);
50  gpio_set_function(UART_RX_PIN, GPIO_FUNC_UART);
51
52  /* =====
53  TODO 4:
54  Configuracion especifica del mecanismo (Polling)
55  Definimos dos acciones criticas derivadas del analisis del correo:
56  ===== */
57
58  /* Deshabilitamos en el NVIC la IRQ del UART0
59  Con esto implementamos la orden del auditor de seguridad "bloquear
60  a nivel fisico cualquier evento o senal asincrona externa" */
61  irq_set_enabled(UART0_IRQ, false); //El hardware no puede interrumpir al
62  CPU
63
64  /* Desactivamos el buffer FIFO del UART.
65  Sin FIFO, cada byte queda disponible de inmediato en el
66  registro de datos, lo que hace al sondeo completamente
67  determinista, un byte listo = una lectura, sin umbrales */
68  uart_set_fifo_enabled(UART_ID, false);
69  }
70
71  int
72  main()
73  {
74  //Llamamos a la configuracion inicial
75  secure_boot_init();
76  //Damos un segundo para que la terminal serial conecte antes de imprimir
77  sleep_ms(1000);
78
79  printf("\n === MODULO DE ARRANQUE SEGURO ===\n");
80  printf("Sistema en modo bloqueo. Esperando clave de %d bytes...\n",
81  KEY_LENGTH);
82  printf("Ingresa los caracteres en la terminal:\n >");
83
84  /* =====
85  TODO 5:
86  Logica principal de recepcion mediante Polling
87
88  Estructura de dos bucles anidados:
89  - Bucle exterior (while bytes_read < KEY_LENGTH):
90  Mantiene al programa capturando hasta completar los 16 bytes.
91  El CPU no puede abandonar este punto; su unica tarea es

```

```

89     esta recepcion, exactamente como describia el correo.
90
91     - Bucle interior (!uart_is_readable):
92     Es el nucleo del Polling, uart_is_readable() consulta el
93     bit RXFE (RX FIFO Empty) del registro de estado del UART
94     Mientras el bit indique "sin datos", se ejecuta
95     tight_loop_contents(), una instruccion NOP de ARM que evita
96     efectos secundarios indeseados durante la espera activa.
97
98     Cuando el byte esta listo (uart_is_readable devuelve true),
99     uart_getc() lo lee del registro de datos del periferico y
100    lo almacena en challenge_key. El printf del eco permite
101    confirmar en la terminal del simulador que cada caracter
102    fue recibido correctamente por el driver.
103    ===== */
104
105    int bytes_read = 0;
106    //Esperamos hasta tener exactamente 16 bytes
107    while (bytes_read < KEY_LENGTH) {
108        /* Busy Waiting:
109        El CPU interroga al hardware en cada ciclo preguntando
110        si ya hay un dato listo en el registro RX del UART */
111        while (!uart_is_readable(UART_ID))
112            tight_loop_contents(); //NOP de ARM para no quemar el CPU
113            innecesariamente
114
115        //Si salimos del bucle interior, significa que hay un byte listo
116        challenge_key[bytes_read] = uart_getc(UART_ID); //Lo leemos y lo
117        guardamos
118
119        //Hacemos un 'eco' hacia la terminal para confirmar la recepcion
120        printf("%c", challenge_key[bytes_read]);
121        bytes_read++;
122    }
123
124    key_received = true; //Confirmamos que terminamos
125
126    // --- EL CODIGO TERMINA AQUI ---
127    //Aseguramos que el arreglo sea un string valido en C para poder
128    imprimirlo
129    challenge_key[KEY_LENGTH] = '\0';
130
131    //Si el flujo del programa llega a esta linea, la recepcion termino
132    printf("\n \n[EXITO] Clave recibida: %s\n", challenge_key);
133    printf("[SISTEMA] Validando integridad...\n");
134    printf("[SISTEMA] Levantando RTOS...\n");
135
136    //Bucle infinito simulando el ciclo de vida normal del sistema operativo
137    while(1)
138        tight_loop_contents();
139
140    return 0;
141 }

```

Código 1: main.c: driver de arranque seguro implementado con Polling

Resultado en el simulador

La salida obtenida al ejecutar el código en Wokwi e ingresando 16 caracteres, fue la siguiente:

```
=== MODULO DE ARRANQUE SEGURO ===  
Sistema en modo bloqueo. Esperando clave de 16 bytes...  
Ingresa los caracteres en la terminal:  
>123456789ABCDEFH  
  
[EXIT0] Clave recibida: 123456789ABCDEFH  
[SISTEMA] Validando integridad...  
[SISTEMA] Levantando RTOS...
```

El programa arrancó correctamente y entró de inmediato en el modo de bloqueo, lo que confirma que las interrupciones del UART quedaron deshabilitadas y que el flujo de ejecución es completamente lineal desde el primer byte, el eco de cada carácter en la línea del prompt muestra que el bucle de Polling detectó y leyó cada byte en el orden en que llegó, sin pérdidas ni reordenamientos.

Al completarse el byte número 16, el bucle exterior terminó de forma natural y el programa avanzó a las líneas de validación e inicio del RTOS, lo que demuestra que la condición de parada `bytes_read < KEY_LENGTH` funcionó con precisión, la clave almacenada en `challenge_key` coincide exactamente con los 16 caracteres ingresados, sin caracteres extra ni faltantes.

Conclusiones

Esta práctica demostró que elegir un mecanismo de **E/S** no es buscar el más eficiente en abstracto, sino el más adecuado para el contexto específico del problema.

El **Módulo 1** mostró que la intuición identifica los rasgos más visibles de cada mecanismo, pero los trata como absolutos, el **Módulo 2** corrigió ese error: el costo del Polling depende de si hay trabajo alternativo pendiente, las Interrupciones tienen mecanismos formales para proteger secciones críticas, y DMA no se descarta por problemas de validación sino por los costos reales de configuración, contención de bus y doble buffering.

El **Módulo 3** confirmó que UART y Polling son aliados naturales, ambos son deterministas, simples y no dependen de subsistemas externos de sincronización, el **Módulo 4** integró todo lo anterior, el análisis del correo redujo un escenario ambiguo a tres restricciones concretas (sin IRQ, flujo lineal, transferencia pequeña) que apuntaron todas al mismo mecanismo. La simulación lo confirmó, el programa capturó exactamente 16 bytes, esperó indefinidamente ante entradas incompletas y nunca fue desviado por señales externas, que es exactamente el comportamiento que el análisis predijo.